

Summary

The final project transitions the single-cycle computer system into an autonomous robot controller. You will modify the VHDL of your 7-segment display hardware to include a command-driven display mode and you will implement multiple “puppy” behaviors in ARMv4 assembly. Your ARMv4 assembly will be allowed to use both leaf and non-leaf subroutines.

Project Management

This project should be completed with the CPE1510 single-cycle computer. **Ensure** you have a backup archive of your working single-cycle computer project **before beginning** the modifications required for this laboratory. This allows you to recover to a known starting state if you make errors in your work.

- **Use Project** → Archive to make a backup archive.

Seven-segment Decoder Modification

Modify your seven-segment decoder so that it interprets the 32-bit input word as a 4-bit command field, an unused field, and a 24-bit data field.

31..28	27..24	23..0
Command	Unused	Data

- Command 0: Standard six-character hexadecimal display of the nibbles on 23 down to 0
- Command 1: Display “Hi” centered on the six-character 7-segment display
- Command 2: Display “Fido” centered on the six-character 7-segment display
- Command 3: Display “Puppy” right-adjusted on the six-character 7-segment display
- Command 4: Display “In Pen” centered on the six-character 7-segment display
- Command 5: Display “Patrol” centered on the six-character 7-segment display
- Command 6: Display “Fetch” right-adjusted on the six-character 7-segment display
- Command 7: Display “Wag” right-adjusted on the six-character 7-segment display
- Command 8: Display “Crawl” right-adjusted on the six-character 7-segment display
- Command 9: Display “Beg” right-adjusted on the six-character 7-segment display

ARMv4 Assembly Main Program

The main assembly program must implement this behavioral control loop.

Main (**_start**)

```

    Display "Hi" on the 7-segment display
    Delay 2 seconds using a delay leaf subroutine
    Display "Wake" on the 7-segment display           // start of tuning circle time
    Delay 2 seconds using a delay leaf subroutine     // give human time to read
    Initialize turn counter to 0
    Spin left

```

Tune: **Repeat**

```

    Increment turn counter
    Until collision sensor activates                 // user stops at full circle
    Display "Fido" on the 7-segment display
    Delay 2 seconds using a delay leaf subroutine

```

Loop: **Repeat**

```

    Read slider switches
    If binary 3 then call subroutine puppy()
    Else if binary 4 then call subroutine pen()
    Else if binary 5 then call subroutine patrol()
    Else if binary 6 then call subroutine fetch()
    Else if binary 7 then call subroutine wag()
    Else if binary 8 then call subroutine crawl()
    Else if binary 9 then call subroutine beg()
    Else turn motors off

```

Forever

ARMv4 Assembly Puppy Subroutine

This subroutine must implement a control algorithm to follow a user's hand.

Puppy

```

    Display "puppy" on the 7-segment display.
    Wait for collision sensor (wake up puppy)

```

Loop: **Repeat**

```

    Forward           // puppy following hand
    Read sensors
    Until no collision sensed any more
    Motors Off

```

Return to Main

ARMv4 Assembly Pen Subroutine

This subroutine must implement a control algorithm to simulate a puppy exploring its pen.

Pen

Counter = 0

Loop: **Repeat**

Display "Pen" on the 7-segment display.

Read sensors

If no black line sensed, then motors on and forward

Else

Back up 1 second

Spin left 90 degrees

Increment Counter

Display Counter on 7-segment display

Wait 1 second

Until Counter = 3

Motors Off

Return to Main

ARMv4 Assembly Patrol Subroutine

This subroutine must implement a control algorithm to simulate a puppy following a black line.

Patrol

Display "Patrol" on the 7-segment display

Wait 2 seconds

Counter = 0

Loop: **Repeat**

Display counter on the 7-segment display

Line follow

Drive over speedbumps and increment counter without stopping

Until counter = 3

Motors Off

Return to Main

ARMv4 Assembly Fetch Subroutine

This subroutine must implement a control algorithm to simulate a puppy fetching a ball.

Fetch

Display "Fetch" on the 7-segment display

Wait for collision

Initialize yards counter to 0

Loop: **Repeat**

Display yards counter on 7-segment display

Drive forward 1 yard (1 yard = 3 feet) (ECBE Tech Support has yard sticks)

Increment yards counter

Until yards counter = 3

Turn around 180 degrees

Drive back decrementing and displaying yards counter along the way

Motors off

Flash all LEDs (on ½ second, off ½ second) four times

Return to Main

ARMv4 Assembly Wag Subroutine

This subroutine must implement a control algorithm to simulate a puppy wagging its tail.

Wag

Display "Wag" on the 7-segment display

Wait for collision

Initialize Wag Counter to 0

Loop: **Repeat**

Display Wag Counter on the 7-segment display

Spin left of forward 45 degrees

Spin to right of forward 45 degrees.

Increment Wag Counter

Until Wag Counter = 10

Motors off

Return to Main

ARMv4 Assembly Crawl Subroutine

This subroutine must implement a control algorithm to simulate a puppy crawling slowly.

Crawl

Display "Crawl" on the 7-segment display

Wait for collision

Initialize Crawl Counter to 0

Loop: **Repeat**

Display Crawl Counter on the 7-segment display

Forward one-half second

Stop one-half second

Increment Crawl Counter

Until Crawl Counter = 10

Motors off

Return to Main

ARMv4 Assembly Beg Subroutine

This subroutine must implement a control algorithm to simulate a puppy begging for a treat.

Beg

Display "Beg" on the 7-segment display

Wait for collision

Initialize Beg Counter to 0

Loop: **Repeat**

Beep two seconds

Spin left in a full circle two times

Beep two seconds

Spin right in a full circle two times

Increment Beg Counter

Until Collision // simulating a treat being dropped in front of the puppy

Motors off

Return to Main

Software Design Principles

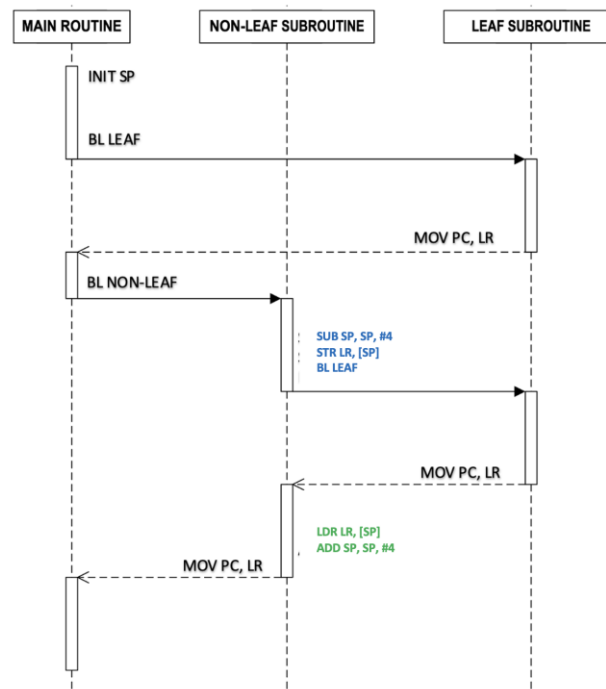
- The circle tuning counter is reused a lot in the behaviors. Use a saved register.
- Angle counts can be calculated from the circle tuning counter.
- Waiting for collision is a common behavior in the software. This is a good subroutine.
- Many other subroutines could be built: **print, motor, blink, degrees**

- Comment your code so that others can understand your work.
- Follow the ARMv4 procedure call standard consistently and correctly. This standard is documented in the register section of your ARMv4 reference card. The caller does not expect a subroutine to preserve R0, R1, R2, R3, and R12. The caller expects a subroutine to do no damage to R4 – R11. The LR, SP, and PC are program flow control registers.

Implementing Non-leaf Subroutines

When a subroutine is called, the return address (PC+4) is stored in the link register (LR), which is register R14. This allows a leaf subroutine to return to its caller using a MOV PC, LR.

Non-leaf subroutines corrupt the link register when they call another subroutine. To ensure corruption does not happen, a subroutine stores its link register value into scratchpad memory called a **stack** before calling the next subroutine. It then retrieves its link register value from the stack before it returns to its caller. This software blueprint illustrates the procedure call models.



The stack pointer (address) is decremented by four bytes to allocate space on the stack, the LR value is stored to it, and then the next subroutine is called. Before the non-leaf subroutine ends, it restores its LR from the stack and deallocates its space by incrementing the SP by four bytes. Your VHDL data memory is a 32-location memory of 32-bit words. This means that you do not have much space to work with! The software blueprint shows the main program initialize the SP to its starting value. Remember, in the ARM ISA, stack addresses grow downward. Thus, you would initialize the SP to address 32 = 0x0000_0020.

Grading

1. Demonstrate one or two puppy subroutines to earn a C-level grade.
2. Demonstrate three to five puppy subroutines to earn a B-level grade.
3. Demonstrate six to seven puppy subroutines to earn an A-level grade.

In all cases, the quality of your assembly language program matters.

1. Use equates (.equ) to define constants like motor command nibbles. For example:

```
.equ FORWARD, 0xF
```

2. Comment your code so that others can understand your work.
3. Did you follow the ARMv4 procedure call standard consistently and correctly. This standard is documented in the register section of your ARMv4 reference card.
 - a. The caller does not expect a subroutine to preserve R0, R1, R2, R3, and R12.
 - b. The caller expects a subroutine to do no damage to R4 – R11.
 - c. The LR, SP, and PC are program flow control registers.

Submissions

- Ensure your assembly language program (.s) is copied into your Quartus project folder.
- Project archives are created using Project → Archive Project... in Quartus.
- Archive your Quartus project with the name:

lastname-firstname-puppy.qar

For example, my file would be named livingston-adam-puppy.qar

- Save the ARM Assembly file running as

lastname-firstname-puppy.s

For example, my file would be named livingston-adam-puppy.s

- Upload your Quartus archive and assembly file to the Canvas Assignment by the **due date** so that Dr. Livingston can examine your work on his machine.
- You must demonstrate your work no later than Friday of Week 15.
- Demonstrations will not be accepted during Final Exam week without prior permission.